

Note that our solution encodes environment dynamics (i.e. game rules) which are unavailable and hard to incorporate in the DQN model. For CLUTRR, we obtain a 25% improvement over baselines, which includes GPT-3-FT, the state-of-the-art large language model fine-tuned on the CLUTRR dataset. Next, for tasks such as HWF and CLEVR, our solutions attain comparable performance, even compared to neurosymbolic baselines NGS-*m*-BS, NSCL, and NS-VQA specifically designed for each task. On Path and Path-X, our solution obtains a 4% accuracy gain over our underlying model CNN and even outperforms a carefully designed transformer based model S4.

The performance of the Scallop solution for each task depends on the chosen provenance structure. As can be seen from Table 3 and Figs. 15–17, although dtkp is generally the best-performing one, each presented provenance is useful, e.g., dtkp for PacMan-Maze and VQAR, damp for less-than (MNIST-R) and Mugen, and dmmp for HWF and CLEVR. Note that under positive Datalog, Scallop’s dtkp is identical to [Huang et al. 2021], allowing us to achieve similar performance. In conclusion, allowing configurable provenance helps tailor our methods to different applications.

6.4 RQ3: Runtime Efficiency

We evaluate the runtime efficiency of Scallop solutions with different provenance structures and compare it against baseline neurosymbolic approaches. As shown in Table 4, Scallop achieves substantial speed-up over DeepProbLog (DPL) on MNIST-R tasks. DPL is a probabilistic programming system based on Prolog using exact probabilistic reasoning. As an example, on sum4, DPL takes 40 days to finish only 4K training samples, showing that it is prohibitively slow to use in practice. On the contrary, Scallop solutions can finish a training epoch (15K samples) in minutes without sacrificing testing accuracy (according to Fig. 15). For HWF, Scallop achieves comparable runtime efficiency, even when compared against the hand-crafted and specialized NGS-*m*-BS method.

Comparing among provenance structures, we see significant runtime blowup when increasing k for dtkp. This is expected as increasing k results in larger boolean formula tags, making the WMC procedure exponentially slower. In practice, we find $k = 3$ for dtkp to be a good balance point between runtime efficiency and reasoning granularity. In fact, dtkp generalizes DPL, as one can set an extremely large $k \geq 2^n$ (n is the total number of input facts) for exact probabilistic reasoning.

6.5 RQ4: Generalizability, Interpretability, and Data-Efficiency

We now consider other important desirable aspects of machine learning models besides accuracy and runtime, such as generalizability on unseen inputs, interpretability of the outputs, and data-efficiency of the training process. For brevity, we focus on a single benchmark task in each case.

We evaluate Scallop’s generalization ability for the CLUTRR task. Each data-point in CLUTRR is annotated with a parameter k denoting the length of the reasoning chain to infer the target kinship relation. To test different solutions’ *systematic generalizability*, we train them on data-points with $k \in \{2, 3\}$ and test on data-points with $k \in \{2, \dots, 10\}$. As shown in Fig. 18, the neural baselines suffer a steep drop in accuracy on the more complex unseen instances, whereas the accuracy of Scallop’s solution degrades more slowly, indicating that it is able to generalize better.

Next, we demonstrate Scallop’s interpretability on the Mugen task. Although the goal of the task is to see whether a video-text pair is aligned, the perceptual model in our method extracts interpretable symbols, i.e., the action of the controlled character at a certain frame. Fig. 19 shows that the predicted (action, mod) pairs perfectly match the events in the video. Thus, our solution not only tells whether a video-text pair is aligned, but also *why* it is aligned.

Lastly, we evaluate Scallop’s data-efficiency on the HWF task, using lesser training data (Table 5). While methods such as NGS-MAPO suffer significantly when trained on less data, Scallop’s testing accuracy decreases slowly, and is comparable to the data-efficiency of the state-of-the-art

Task	Scallop				Baseline
	dtmp	damp	dtkp-3	dtkp-10	
sum2	34	88	72	185	21,430 (DPL)
sum3	34	119	71	1,430	30,898 (DPL)
sum4	34	154	77	4,329	timeout (DPL)
less-than	35	42	34	43	2,540 (DPL)
not-3-or-4	37	33	33	34	3,218 (DPL)
HWF	89	107	120	8,435	79 (NGS- <i>m</i> -BS)
CLEVR	1,964	1,618	2,325	timeout	–

Table 4. Runtime efficiency comparison on selected benchmark tasks. Numbers shown are average training time (sec.) per epoch. Our variants attaining the best accuracy are indicated in bold.



Fig. 19. The predicted most likely (action, mod) pair for example video segments from MUGEN dataset.

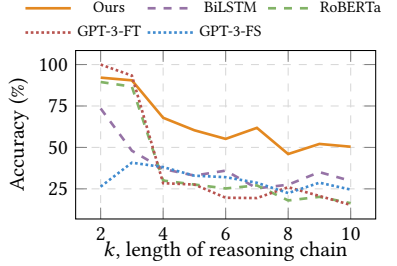


Fig. 18. Systematic generalizability on CLUTRR dataset.

%Train	Scallop	NGS		
	dtkp-5	RL	MAPO	<i>m</i> -BS
100%	97.85	3.4	71.7	98.5
50%	95.7	3.6	9.5	95.7
25%	92.95	3.5	5.1	93.3

Table 5. Testing accuracy of various methods on HWF when trained with only a portion of the data. Numbers are in percentage (%).

neurosymbolic NGS-*m*-BS method. PacMan-Maze task also demonstrates Scallop’s data-efficiency, as it takes much less training episodes than DQN does, while achieving much higher success rate.

6.6 RQ5: Analysis of Failure Modes

Compared to purely neural models, Scallop solutions provide more transparency, allowing programmers to debug effectively. By manually checking the interface relations, we observed that the main source of error lies in inaccurate predictions from the neural components. For example, the RoBERTa model for CLUTRR correctly extracts only 84.69% of kinship relations. There are two potential causes—either the neural component is not powerful enough, or our solution is not providing adequate supervision to train it. The former can be addressed by employing better neural architectures or more data. The latter can be mitigated in different ways, such as tuning the selected provenance or incorporating integrity constraints (discussed in Section 3.2) into training and/or inference. For instance, in PacMan-Maze, a constraint that “there should be no more than one *goal* in the arena” helps to improve robustness.

7 RELATED WORK

We survey related work in four different but overlapping domains: provenance reasoning, Datalog and logic programming, probabilistic and differentiable programming, and neurosymbolic methods.

Relational Algebra and Provenance. Relational algebra is extensively studied in databases [Abiteboul et al. 1995], and extended with recursion [Jachiet et al. 2020] in Datalog. The provenance semiring framework [Green et al. 2007] was first proposed for positive relational algebra (RA^+) and later extended with difference [Amsterdamer et al. 2011] and fixed-point [Dannert et al. 2021]. It is deployed in variants of Datalog [Khamis et al. 2021] and supports applications such as program synthesis [Si et al. 2019]. Scallop employs an extended provenance semiring framework and demonstrates its application to configurable differentiable reasoning.

Datalog is a declarative logic programming language where rules are logical formulas. Despite being less expressive than Prolog, it is widely studied in both the programming language and

database communities for language extensions and optimizations [Khamis et al. 2021]. A variety of Datalog-based systems has been built for program analysis [Scholz et al. 2016] and enterprise databases [Aref et al. 2015]. Scallop is also a Datalog-based system that extends [Huang et al. 2021] with numerous language constructs such as negation and aggregation.

Probabilistic Programming is a paradigm for programmers to model distributions and perform probabilistic sampling and inference. Such systems include ProbLog [Dries et al. 2015], Pyro [Bingham et al. 2018], Turing [Ge et al. 2018], and PPL [van de Meent et al. 2018]. When integrated with modern ML systems, they are well suited for statistical modeling and building generative models. Scallop also supports ProbLog-style exact probabilistic inference as one instantiation of our provenance framework. But advanced statistical sampling and generative modeling is not yet supported and left for future work.

Differentiable Programming (DP) systems seek to enable writing code that is differentiable. Common practices for DP include symbolic differentiation and automatic differentiation (auto-diff) [Baydin et al. 2015], resulting in popular ML frameworks such as PyTorch [Paszke et al. 2019], TensorFlow [Abadi et al. 2015], and JAX [Bradbury et al. 2018]. However, most of these systems are designed for real-valued functions. Scallop is also a differentiable programming system, but with a focus on programming with discrete, logical, and relational symbols.

Neurosymbolic Methods have emerged to incorporate symbolic reasoning into existing learning systems. Their success has been demonstrated in a number of applications [Chen et al. 2020; Li et al. 2020; Mao et al. 2019; Minervini et al. 2020; Wang et al. 2019; Xu et al. 2022; Yi et al. 2018]. Similar to [Li et al. 2020; Mao et al. 2019; Yi et al. 2018], we focus primarily on solutions involving perception followed by symbolic reasoning. Other ways of incorporating symbolic knowledge include *semantic loss* [Xu et al. 2018, 2022], *program synthesis* [Chen et al. 2021a; Shah et al. 2020], and invoking large language models (LLMs) [Cheng et al. 2022; Zelikman et al. 2023]. Most aforementioned neurosymbolic methods build their own domain-specific language or specialized reasoning components, many of which are programmable in Scallop and are instantiations of our provenance framework [Chen et al. 2020; Mao et al. 2019; Xu et al. 2018, 2022]. TensorLog [Cohen et al. 2017], DPL [Manhaeve et al. 2021], and [Huang et al. 2021] can be viewed as unified neurosymbolic frameworks of varying expressivity. Scallop is inspired by DPL, and additionally offers a more scalable, customizable, and easy-to-use language and framework.

8 CONCLUSION

We presented Scallop, a neurosymbolic programming language for integrating deep learning and logical reasoning. We introduced a declarative language and a reasoning framework based on provenance computations. We showed that our framework is practical by applying it to a variety of machine learning tasks. In particular, our experiments show that Scallop solutions are comparable and even supersede many existing baselines.

In the future, we plan to extend Scallop in three aspects: 1) Supporting more machine learning paradigms, such as generative modeling, open domain reasoning, in-context learning, and adversarial learning. 2) Further enhancing the usability, efficiency, and expressiveness of Scallop’s language and framework. We intend to provide bindings to other ML frameworks such as TensorFlow and JAX, leverage hardware such as GPUs to accelerate computation, and support constructs such as algebraic data types. 3) Applying Scallop to real-world and safety-critical domains. For instance, we intend to integrate it with the CARLA driving simulator [Dosovitskiy et al. 2017] to specify soft temporal constraints for autonomous driving systems. We also intend to apply Scallop in the medical domain for explainable disease diagnosis from electronic health records (EHR) data.